

QUESTION 4

Title:

Crypto Price Sorting Using Quick Sort

Aim:

To analyze the use of Quick Sort in sorting cryptocurrency prices efficiently and evaluate its performance, pivot selection, and optimization techniques.

Problem Understanding:

A cryptocurrency platform tracks thousands of coin prices that change continuously. To display prices in sorted order for analysis and trading, an efficient sorting algorithm is required.

Quick Sort is a Divide and Conquer algorithm that partitions data around a pivot element and recursively sorts the partitions. It is widely used because of its fast average-case performance.

Algorithm:

- Select a pivot element.
- Divide the prices into two groups:
 - Elements smaller than the pivot.
 - Elements greater than the pivot.
- Recursively sort both groups.
- Combine the sorted groups with the pivot.
- Display the sorted prices.

Pseudocode:

QUICKSORT(arr)

IF length(arr) <= 1

 RETURN arr

pivot = first element

left = elements $\leq$ pivot

right = elements $>$ pivot

RETURN QUICKSORT(left) + pivot + QUICKSORT(right)

Source Code:

```
def quicksort(arr):  
    if len(arr) <= 1:  
        return arr  
  
    pivot = arr[0]  
  
    left = [x for x in arr[1:] if x <= pivot]  
    right = [x for x in arr[1:] if x > pivot]  
  
    return quicksort(left) + [pivot] + quicksort(right)  
  
prices = [50, 20, 80, 10, 40]  
  
print("Original Prices:", prices)  
print("Sorted Prices:", quicksort(prices))
```

Sample Output:

Original Prices: [50, 20, 80, 10, 40]

Sorted Prices: [10, 20, 40, 50, 80]

Analysis:

Divide and Conquer Principle

Divide

The list is divided around a pivot element.

Example:

[50, 20, 80, 10, 40]

Pivot = 50

Left = [20, 10, 40]

Right = [80]

Conquer

Recursively sort the left and right partitions.

Combine

Merge the sorted partitions with the pivot.

Pivot Selection

Good Pivot

Creates balanced partitions and improves performance.

Bad Pivot

Creates uneven partitions and may increase execution time.

Fluctuating Data

Cryptocurrency prices change frequently. Quick Sort can quickly reorder updated prices, making it suitable for dynamic systems.

Optimizations

- Random Pivot Selection
- Median-of-Three Pivot
- Parallel Quick Sort
- Hybrid Sorting Techniques

Time Complexity

Case	Complexity
-------------	-------------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Case	Complexity
------	------------

Worst Case	$O(n^2)$
------------	----------

Space Complexity

$O(\log n)$

Result

The study shows that Quick Sort is an effective sorting algorithm for cryptocurrency price tracking systems. Its divide-and-conquer approach provides fast average-case performance, while proper pivot selection helps maintain efficiency. Therefore, Quick Sort is suitable for sorting large volumes of dynamic cryptocurrency data.